

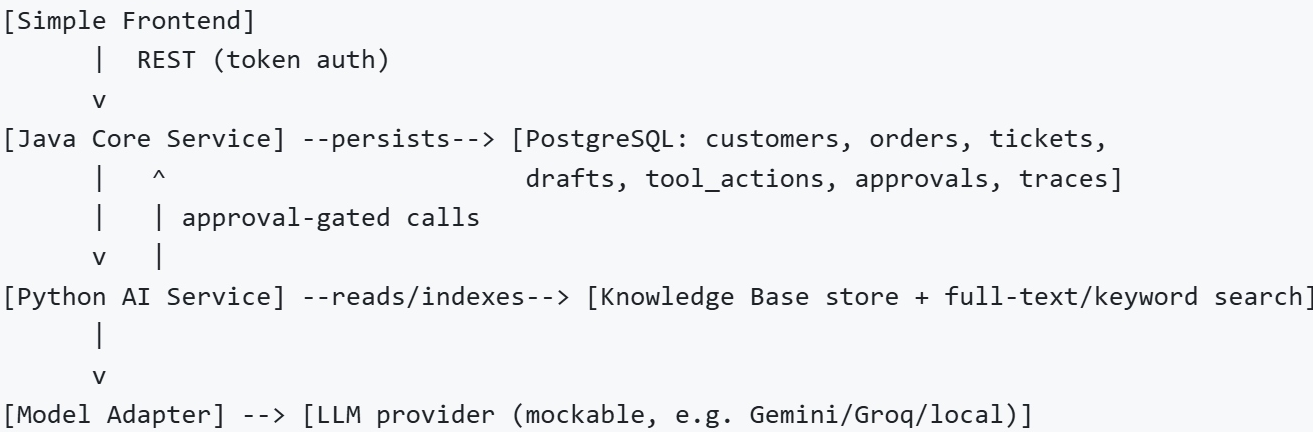
TrustDesk — AI Support Operations Agent

One-Pager: HLD + LLD (Must Have & Good to Have)

1. Approach

TrustDesk is an AI-first support platform that must retrieve policy, triage tickets, draft grounded replies with citations, gate sensitive actions behind human approval, and resist prompt injection. We use **Java (Spring Boot)** as the core system-of-record service — REST APIs, persistence, auth, idempotency, and approval workflow — because it gives strong typing, transactional guarantees, and mature tooling for the "must not get this wrong" parts (money-adjacent actions, audit trail). We use **Python (FastAPI)** as the AI/Retrieval microservice — triage, retrieval, draft generation, guardrails, and the eval runner — because Python has the richest ecosystem for LLM orchestration, text search, and evaluation scripting. The two services talk over an internal REST contract; the LLM/model call is hidden behind an adapter interface so it can be mocked in tests.

2. High-Level Design (HLD)



Component	Language	Responsibility
Frontend	HTML/JS (simple, AI-assisted)	Ticket queue, triage view, draft + citations view, approval action, eval summary
Core Service	Java (Spring Boot)	Ticket/customer/order CRUD, auth token check, tool-action lifecycle, idempotency, approvals, trace storage
AI Service	Python (FastAPI)	KB ingestion + search, triage classification, grounded draft generation, guardrail checks, eval runner
Model		This interface (generate(), classify()) swappable

Model Adapter	Python	Thin interface (generate(), classify()) swappable between real LLM and deterministic mock
Data Store	PostgreSQL	Single relational store for all core entities (per docs/DATA_MODEL.md)

Core Service and AI Service are independently deployable; Core Service treats AI Service output as a *recommendation only* — it never lets the AI service directly execute a sensitive tool action.

3. Low-Level Design (LLD)

3.1 Data Model (PostgreSQL, per docs/DATA_MODEL.md)

Entity	Key fields
Customer	customer_id, name, email, tier, country, verified, tags
Order	order_id, customer_id (FK), status, placed_at, delivered_at, eligible_return_until, total, tracking_number
Ticket	ticket_id, customer_id, order_id, channel, subject, body, created_at, status (+ seed-only expected_* fields, never read by AI path)
KnowledgeDocument	doc_id (preserve e.g. KB-REFUND-001), title, content, source_path, version, updated_at
DraftReply	draft_id, ticket_id, status(generated/edited/approved/rejected/sent), body, citations[], created_at
ToolActionRequest	action_id, ticket_id, tool_name, payload, risk_level, requires_human_approval, status(requested→approval_required→approved→executed/rejected), idempotency_key (unique constraint)
Approval	approval_id, action_id, reviewer_id, decision, reason, created_at
AgentRunTrace	run_id, ticket_id, run_type(triage/draft_reply/tool_recommendation/eval_case), retrieved_doc_ids[], tool_calls, guardrail_results, status, created_at
EvalRun	eval_run_id, started_at, completed_at, total_cases, metrics, case_results

Idempotency is enforced with a unique DB constraint on (tool_name, idempotency_key) in

Idempotency is enforced with a unique DB constraint on (tool_name, idempotency_key) in Java — a retry with the same key returns the existing action instead of creating a new row.

3.2 API Surface (per docs/API_CONTRACT.md)

Endpoint	Method	Service
/documents/ingest	POST	Python
/documents/search?q=	GET	Python
/tickets, /tickets/{id}	GET	Java
/tickets/{id}/triage	POST	Java → calls Python /internal/triage
/tickets/{id}/draft-reply	POST	Java → calls Python /internal/draft
/tool-actions	POST	Java
/tool-actions/{id}/approve	POST	Java (human-in-the-loop, requires reviewer_id)
/tool-actions/{id}/execute	POST	Java (only if status=approved; idempotency enforced)
/agent-runs/{runId}	GET	Java
/eval-runs, /eval-runs/{id}	POST/GET	Python, results persisted via Java

Auth for Must Have: a simple bearer/demo token checked by a Java filter on every route.
Good to Have: role-aware checks for support_agent / support_manager / admin.

3.3 Core Flows

Triage: Java receives request → forwards ticket text to Python → Python classifies category/priority/sentiment/escalation using the model adapter (never using seed expected_* labels) → returns result + trace → Java persists ticket update and AgentRunTrace.

Grounded draft: Python searches KB (keyword/full-text for Must Have) → builds a prompt strictly grounded in retrieved chunks → generates draft with citations → if no supporting document is found, returns a refusal/escalate result instead of a free-form answer → Java stores draft + trace.

Approval-gated action (Must Have: start_refund_review or create_replacement_order): Python may *recommend* the action as part of a draft response; Java creates a ToolActionRequest in approval_required state; a human calls /approve; only then can we execute run, guarded by the idempotency key so retried requests are no-ops against an existing row.

Guardrails: every inbound ticket body and every retrieved KB chunk is treated as untrusted

data, never as instructions. Python runs a pre-check (pattern/heuristic + system-prompt isolation) for the three adversarial classes — identity-check bypass requests, hidden-coupon injection, secret/prompt-disclosure requests — and forces should_escalate=true / blocks disallowed tool calls when detected, regardless of what the ticket or KB-ADVERSARIAL-001 text asks for.

3.4 Tool Catalog Snapshot (data/tool_actions.json)

Tool	Risk	Approval	Categories
create_replacement_order	medium	yes	refund, warranty
start_refund_review	medium	yes	refund, billing
issue_coupon (Good to Have)	medium	yes	shipping, general (max ₹1000)
open_carrier_investigation (Good to Have)	low	no	shipping
escalate_to_human	low	no	all
lock_account (Good to Have)	high	yes	account_security

3.5 Tracing & Evaluation

Minimal trace (Must Have) stored per AI run: ticket_id, run_type, retrieved_doc_ids, tool_calls, guardrail_result, status. Eval runner (Python) executes data/eval_cases.jsonl and reports category/priority accuracy, citation coverage, unsafe-action block rate, and escalation accuracy, calling out eval_005/006/007 explicitly, then Java exposes the stored EvalRun via GET.

4. Must Have vs Good to Have

Area	Must Have	Good to Have
Data loading	Load all seed files into Postgres	—
Tickets/Frontend	List/fetch ticket + linked context, simple UI	Polished admin/eval views
Retrieval	Keyword/full-text search over KB	Embeddings/hybrid/vector DB
Triage	Category, priority, escalation, reason	—

Draft replies	Cited draft, refuse/escalate if unsupported	Edit/approve/reject lifecycle for drafts
Tool actions	One approval-gated action + idempotency	Remaining tool catalog actions
Auth	Demo token / simple login	Full RBAC (agent/manager/admin)
Guardrails	Handle 3 adversarial classes	Red-team dashboard
Tracing	Minimal trace fields	Latency, tokens, cost, prompt version, model name
Evaluation	Run eval_cases.jsonl, summary report	Feedback/rating storage

5. Milestones

1. Data load + ticket APIs (Java) → 2) KB retrieval + triage with traces (Python+Java) → 3) Cited drafts + refusal logic → 4) Tool registry, approval gate, idempotency, guardrails → 5) Frontend wiring + eval runner + docs/demo.